

EXAMEN DE FIN DE MODULE : PROGRAMMATION JAVASCRIPT

Niveaux Intermédiaire et Avancé

Durée : 3 heures Total : 100 points

PARTIE I : QUESTIONS THÉORIQUES (30 points)

1. Expliquez la différence entre les méthodes `call()`, `apply()` et `bind()` en JavaScript. Donnez un exemple pratique pour chacune. (6 points)
 2. Qu'est-ce que l'hoisting en JavaScript ? Expliquez comment il fonctionne pour les variables déclarées avec `var`, `let` et `const`, ainsi que pour les déclarations de fonctions. (5 points)
 3. Expliquez le concept de fermeture (closure) en JavaScript. Comment et pourquoi les utilise-t-on ? Donnez un exemple pratique. (5 points)
 4. Comparez et contrastiez les Promesses et `async/await` en JavaScript. Quels sont les avantages de chaque approche ? (5 points)
 5. Décrivez les différentes méthodes pour gérer l'héritage en JavaScript (prototypes, classes ES6, mixins). Quels sont les avantages et inconvénients de chaque approche ? (5 points)
 6. Expliquez le concept d'Event Loop en JavaScript. Comment fonctionne-t-il avec les opérations synchrones et asynchrones ? (4 points)
-

PARTIE II : EXERCICES PRATIQUES (40 points)

Exercice 1 : Manipulation d'objets et prototypes (10 points)

Créez une structure de classes/prototypes pour un système de gestion de bibliothèque avec les entités suivantes :

- `Média` (classe parent)
- `Livre` (hérite de `Média`)
- `DVD` (hérite de `Média`)
- `Utilisateur`
- `Bibliothèque` (peut contenir des médias et gérer les emprunts)

Chaque média doit avoir un titre, un ID unique, et un statut (disponible/emprunté). Les utilisateurs doivent pouvoir emprunter et rendre des médias. La bibliothèque doit pouvoir lister tous les médias disponibles et empruntés.

Exercice 2 : Programmation asynchrone (10 points)

Implémentez une fonction qui simule le chargement de données à partir de trois sources différentes (simulez ces sources avec `setTimeout`). La fonction doit :

1. Charger les données des trois sources en parallèle
2. Afficher un message lorsque chaque source a terminé son chargement
3. Une fois toutes les données chargées, les combiner dans un seul objet et le retourner
4. Gérer correctement les erreurs potentielles

Implémentez cette fonction deux fois : une fois avec les Promesses, et une fois avec `async/await`.

Exercice 3 : Manipulation du DOM et performances (10 points)

Écrivez une fonction qui crée un tableau dynamique à partir d'un tableau d'objets fourni en paramètre. Votre solution doit :

1. Créer un tableau HTML avec en-têtes basés sur les propriétés des objets
2. Permettre le tri du tableau en cliquant sur les en-têtes (tri ascendant/descendant)
3. Implémenter une pagination (10 éléments par page)
4. Optimiser les performances pour gérer efficacement de grands ensembles de données (1000+ éléments)
5. Permettre la recherche/filtrage dans le tableau

Exercice 4 : Design Patterns (10 points)

Implémentez un système de gestion d'état simple pour une application en utilisant le pattern Observer ou le pattern Publish/Subscribe.

Votre implémentation doit :

1. Permettre à différents composants de s'abonner à des changements d'état
2. Notifier tous les composants abonnés lorsque l'état change
3. Permettre aux composants de se désabonner
4. Inclure une méthode pour déclencher un changement d'état

Démontrez l'utilisation de votre système avec au moins deux composants qui réagissent aux changements d'état.

PARTIE III : PROJET PRATIQUE (30 points)

Développement d'une application Single Page

Créez une application de liste de tâches (Todo List) qui utilise les fonctionnalités avancées de JavaScript. L'application doit être construite sans frameworks externes (pas de React, Vue, etc.), en utilisant uniquement du JavaScript vanilla.

Fonctionnalités requises :

1. Gestion des tâches (8 points)

- Ajouter, supprimer, et marquer les tâches comme complétées
- Éditer les tâches existantes
- Catégoriser les tâches (travail, personnel, urgent, etc.)
- Ajouter des dates d'échéance

2. Persistance des données (6 points)

- Sauvegarder les tâches dans le localStorage
- Implémenter une simulation d'API avec des promesses pour la sauvegarde/récupération

3. Interface utilisateur (8 points)

- Interface réactive sans rechargement de page
- Animations pour l'ajout/suppression de tâches
- Mode sombre/clair
- Design responsive

4. Fonctionnalités avancées (8 points)

- Filtrage et recherche des tâches
- Fonction de glisser-déposer pour réorganiser les tâches
- Statistiques sur les tâches (complétées vs en cours)
- Notifications pour les tâches en retard (simulées)

Critères d'évaluation :

- Qualité et organisation du code
 - Utilisation appropriée des concepts avancés de JavaScript
 - Performance et optimisation
 - Gestion des erreurs
 - Interface utilisateur et expérience utilisateur
-

BONUS (10 points supplémentaires)

Implémentez l'une des fonctionnalités suivantes dans votre projet :

1. Un système de plugins permettant d'étendre les fonctionnalités de l'application
2. Une fonctionnalité de synchronisation avec une API externe (simulée)
3. Un système d'internationalisation (i18n) pour supporter plusieurs langues
4. Des tests unitaires pour les principales fonctionnalités

Bonne chance !